In [1]:
```
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.getcwd())))


from query_helper import get_connection, run_query

conn = get_connection()
```

```
Connection to database successful.
```

In [2]:
```
"""
Retrieve the top 10 departments by total visit volume.
"""

_ = run_query(conn, """
    SELECT department, COUNT(*) as total_visits
    FROM visits
    GROUP BY department
    ORDER BY total_visits DESC
    LIMIT 10
""", "Top 10 Departments by Visit Volume")
```

```
============================================================
  Top 10 Departments by Visit Volume
============================================================

📋 Query Plan:
   SCAN visits USING COVERING INDEX idx_visits_department
   USE TEMP B-TREE FOR ORDER BY

📊 Results (6 rows):
```

	department	total_visits
0	General	4228
1	ER	4220
2	Neurology	4165
3	Orthopedics	4164
4	Cardiology	4159
5	ICU	4064

In [3]:
```
"""
Identify the top 5 departments with the highest average length of stay.
"""

_ = run_query(conn, """
    SELECT department, ROUND(AVG(length_of_stay_hours), 2) as avg_stay
    FROM visits
    GROUP BY department
    ORDER BY avg_stay DESC
```

```
    LIMIT 5
""", "Top 5 Departments by Average Length of Stay")
```

```
============================================================
  Top 5 Departments by Average Length of Stay
============================================================
```

📋 Query Plan:
```
SCAN visits USING INDEX idx_visits_department
USE TEMP B-TREE FOR ORDER BY
```

📊 Results (5 rows):

	department	avg_stay
0	Neurology	19.72
1	Orthopedics	19.66
2	Cardiology	19.60
3	ER	19.53
4	General	19.43

In [4]:
```
"""
Find the percentage of High Risk visits per department.
"""

_ = run_query(conn, """
    SELECT department,
           COUNT(*) as total_visits,
           SUM(CASE WHEN risk_score = 'High' THEN 1 ELSE 0 END) as high_risk_visits,
           ROUND(100.0 * SUM(CASE WHEN risk_score = 'High' THEN 1 ELSE 0 END) / COUNT(*), 2) as high_risk_percentage
    FROM visits
    GROUP BY department
    ORDER BY high_risk_percentage DESC
""", "Percentage of High Risk Visits per Department")
```

```
============================================================
  Percentage of High Risk Visits per Department
============================================================
```

📋 Query Plan:
```
SCAN visits USING INDEX idx_visits_department
USE TEMP B-TREE FOR ORDER BY
```

📊 Results (6 rows):

	department	total_visits	high_risk_visits	high_risk_percentage
0	ICU	4064	845	20.79
1	ER	4220	872	20.66
2	Neurology	4165	846	20.31
3	Orthopedics	4164	842	20.22
4	General	4228	839	19.84
5	Cardiology	4159	790	18.99

In [5]:
```
"""
Determine the average number of visits per patient by city.
"""

_ = run_query(conn, """
    SELECT p.city,
           COUNT(*) as total_visits,
           COUNT(DISTINCT v.patient_id) as unique_patients,
           ROUND(COUNT(*) * 1.0 / COUNT(DISTINCT v.patient_id), 2) as avg_visits_per_patient
    FROM visits v
    JOIN patients p ON v.patient_id = p.patient_id
    GROUP BY p.city
    ORDER BY avg_visits_per_patient DESC
""", "Average Number of Visits per Patient by City")
```

```
============================================================
  Average Number of Visits per Patient by City
============================================================

📋 Query Plan:
   SCAN p USING COVERING INDEX idx_patients_city
   SEARCH v USING COVERING INDEX idx_visits_patient_id (patient_id=?)
   USE TEMP B-TREE FOR count(DISTINCT)
   USE TEMP B-TREE FOR ORDER BY

📊 Results (6 rows):
```

	city	total_visits	unique_patients	avg_visits_per_patient
0	Pune	4221	824	5.12
1	Hyderabad	4370	864	5.06
2	Bangalore	4205	837	5.02
3	Chennai	3975	792	5.02
4	Mumbai	4122	821	5.02
5	Delhi	4107	829	4.95

In [6]:
```
"""
Identify doctors handling the highest number of High Risk visits.
"""
```

```
_ = run_query(conn, """
    SELECT doctor_id,
           COUNT(*) as total_visits,
           SUM(CASE WHEN risk_score = 'High' THEN 1 ELSE 0 END) as high_risk_visits
    FROM visits
    GROUP BY doctor_id
    ORDER BY high_risk_visits DESC
    LIMIT 10
""", "Doctors Handling the Highest Number of High Risk Visits")
```

```
============================================================
  Doctors Handling the Highest Number of High Risk Visits
============================================================
```

📋 Query Plan:

```
SCAN visits USING INDEX idx_visits_doctor_id
USE TEMP B-TREE FOR ORDER BY
```

📊 Results (10 rows):

	doctor_id	total_visits	high_risk_visits
0	174	264	71
1	198	252	69
2	169	279	68
3	177	266	67
4	105	250	65
5	135	261	65
6	180	290	64
7	188	285	64
8	131	266	62
9	108	242	61